CSD415 PROJECT PHASE – I

DESIGN DOCUMENT – PART II

PROJECT TITLE

Submitted by

Aiswarya R	IDK23CS006
Niranjana K D	IDK23CS045
Vimal Vijayan	IDK23CS063
Vishnu P S	IDK23CS064

Project Guide

Dr. Rani Koshy

GOVERNMENT ENGINEERING COLLEGE, IDUKKI

Department of Computer Science and Engineering

Academic Year 2026–2027

Contents

2 Detailed System Design **1**

2.1 Component Design . . . 1

2.1.1 Data Acquisition Module . . . 1

2.1.2 Preprocessing Module . . . 4

2.1.3 Feature Extraction Module . . . 7

2.1.4 Machine Learning Model Module . . . 12

2.1.5 Prediction and Decision Module . . . 14

2.1.6 Database Module . . . 15

2.1.7 User Interface Module . . . 16

2.2 Functional Design . . . 17

2.2.1 Use Case Diagram . . . 18

2.2.2 Activity Diagram . . . 19

2.2.3 Sequence Diagram . . . 20

2.2.4 Class Diagram . . . 20

2.3 Algorithm / Process Design . . . 22

2.3.1 Enrollment and Training (Offline) . . . 22

2.3.2 Continuous Authentication (Runtime) . . . 22

2.3.3 Trust-Score Evaluation . . . 23

2.3.4 Adaptive Authentication Decision . . . 24

2.3.5 Complexity and Latency Considerations . . . 24

2.4 Database Design . . . 24

2.4.1 Entities, Attributes and Keys . . . 24

2.4.2 Relationships . . . 25

2.5 User Interface Design . . . 25

2.5.1 Design Principles . . . 27

2.5.2 Screen Overview . . . 27

2.5.3 Login Screen . . . 27

2.5.4 Simulated Banking Dashboard . . . 27

2.5.5 Authentication Status Display . . . 28

2.5.6 Step-Up Authentication Screen . . . 28

2.5.7 Session Termination Screen . . . 28

2.5.8 Collection-Mode Screens . . . 28

2.5.9 Status and Open Items . . . 28

2.6 Hardware and Software Design . . . 28

2.6.1 Hardware Design . . . 28

2.6.2 Software Design . . . 29

2.6.3 Deployment Arrangement . . . 29

2.6.4 Unresolved Implementation Choices . . . 29

2.6.5 Status . . . 29

List of Figures

2.1 Data acquisition, preprocessing and feature extraction pipeline of the proposed system. Blue blocks hold raw timestamped observations, green blocks preprocessed data, yellow blocks extracted features; red blocks show how invalid or insufficient observations are handled. Identifiers B04–B07, C2–C5, A03–A05 and K3 refer to Figure 1.3 and Tables 1.2–1.4 of Part I. Window parameters and the ownership of standardization are unresolved (U). 11

2.2 Use case diagram of the proposed behavioral biometric-based continuous authentication system for secure mobile banking. 18

2.3 Activity diagram of the continuous authentication workflow after initial login. 19

2.4 Sequence diagram of continuous authentication during an active banking session. 20

2.5 Class diagram of the proposed logical structure of the continuous authentication system. . 21

2.6 Entity–relationship diagram of the continuous authentication system. 26

List of Tables

2.1 Input, method, processing and output of the Data Acquisition Module. 2
2.2 Acquisition of each behavioral modality. 2
2.3 Handling of unavailable or interrupted acquisition (all entries proposed). 4
2.4 Stages of the Preprocessing Module (all stages proposed; status in last column). 5
2.5 Preprocessing parameters that are not yet established. 6
2.6 Candidate keystroke features. 7
2.7 Candidate touch features. 8
2.8 Candidate swipe features. 8
2.9 Candidate motion features (per sensor). 9
2.10 Candidate features grouped by dependency. 9
2.11 Logical structure of the feature record (A05). 9
2.12 Logical interfaces of the data-processing modules. 10
2.13 Recommended allocation of the data-processing operations (requires approval). 10
2.14 Machine Learning Model Module: input, processing, output and interfaces. 12
2.15 Use of HMOG data in each partition. 12
2.16 Roles of the models in continuous authentication. 13
2.17 Prediction and Decision Module: inputs, processing, outputs and interfaces. 14
2.18 Trust-score-based authentication decision rules. 15
2.19 Database Module: input, processing, output and interfaces. 15
2.20 Responsibilities of the User Interface Module (proposed). 16
2.21 Inputs and outputs of the User Interface Module. 17
2.22 Diagrams of the functional design. 18
2.23 Responsibilities of the classes in Figure 2.5. 21
2.24 Entities, attributes and keys. 24
2.25 Proposed screens of the mobile banking application prototype. 27
2.26 Hardware requirements. 29
2.27 Software requirements. 30
2.28 Open deployment options (unresolved). 30
2.29 Hardware and software items that remain to be decided. 30

2 Detailed System Design

2.1 Component Design

This section specifies the modules of the proposed system in the order in which data flows through them: Data Acquisition (Section 2.1.1), Preprocessing (2.1.2), Feature Extraction (2.1.3), Machine Learning Model (2.1.4), Prediction and Decision (2.1.5), Database (2.1.6) and User Interface (2.1.7). Component, block and arrow identifiers (B, C, A, K) refer to Part I, Figure 1.3 and Tables 1.2–1.4. Status markers follow Part I: C confirmed, S supported by the Literature Survey, P proposed, U unresolved; TBD marks a value that is not yet fixed. The document describes a proposed design; nothing in it has been implemented or verified.

2.1.1 Data Acquisition Module

The Data Acquisition Module (component C2, blocks B04a–B04d in Figure 1.3) is the entry point of the data-processing path. It records timestamped behavioral events and sensor readings produced while a user interacts with the Android banking prototype and passes them, without interpretation, to the Preprocessing Module (C3). It computes no features, applies no scaling and makes no authentication decision. The selected primary source for offline development, training and evaluation is the publicly available HMOG dataset. The mobile prototype will separately capture live behavioral events at runtime using compatible preprocessing and feature definitions. Additional public datasets may be incorporated later if needed, after their licenses, modality coverage and schema compatibility have been checked.

Operating modes

The system has two data paths. They share the same logical record definitions and the downstream preprocessing and feature schema (shared definitions K3 in Figure 1.3), but the sources differ.

- *Offline dataset path.* HMOG session files are imported from the downloaded public dataset, parsed into the project's canonical record schema, and passed through the same preprocessing and feature-extraction stages used by the runtime system. Participant and session identifiers are retained as dataset metadata for partitioning and evaluation, not as model input features.
- *Runtime authentication path.* After the PIN/fingerprint login (A01), the Android banking prototype captures live events and sensor readings and streams records to the Preprocessing Module (A03). Runtime records do not include a ground-truth identity label; the claimed user is associated with the authenticated session.

Selected dataset and integration strategy

HMOG (Hand Movement, Orientation, and Grasp) is the primary dataset selected for offline model development. The dataset was collected from 100 volunteers using smartphone reading, text-production and map-navigation tasks, with sessions performed while sitting or walking. Its published data categories include accelerometer, gyroscope, magnetometer, raw touch events, tap, scale, scroll and fling gestures, and virtual-keyboard key presses; session metadata is also supplied. The dataset is distributed for non-commercial educational and research use under the provider's terms, so the project must follow those terms, acknowledge the dataset and its associated publications, and must not redistribute the dataset. The dataset description and required acknowledgement are available at `https://hmog-dataset.github.io/hmog/`; the associated dataset paper is Yang et al., ACM SenSys 2014, DOI: 10.1145/2668332.2668366.

HMOG was not collected from our banking prototype, so it will be used as an offline behavioral dataset rather than treated as live banking-app telemetry. Before model training, the team must inspect the downloaded files and metadata, confirm timestamp units and field definitions, map supported HMOG categories to the canonical schema, and document unavailable fields. In particular, the initial project feature set targets keystroke, touch/swipe and accelerometer/gyroscope behavior; magnetometer data may be retained for later experiments only if the team approves extending the modality scope. HMOG key-press records and touch/gesture records must not be assumed to contain every field required by the proposed runtime design (for example, matched key press/release events or pressure); feature groups

depending on unavailable fields will be omitted consistently from training and runtime. Any additional dataset will be treated as a supplementary source and processed only after a separate compatibility audit.

Input, acquisition method, processing and output

Table 2.1 summarizes the module in the format of the design guidelines.

Table 2.1: Input, method, processing and output of the Data Acquisition Module.

Item	Specification	Status
Input	Offline: HMOG CSV files and session metadata; runtime: session-start event after login (A01), prototype touch/key events and accelerometer/gyroscope readings (A02)	C / P
Method	Passive listeners registered at session start and released at session end; no deliberate user action; no recording of typed characters, credentials or amounts (Part I, Section 1.5.1)	C / P
Processing	(i) stamp each record on one session clock; (ii) attach `session_id`, `task_context`, `stream_type`, `event_type` and the device/sensor descriptor; (iii) assign a non-character `key_slot` to key events; (iv) emit capture-state markers (start, stop, gap); (v) forward records in time-ordered batches	P
Output	Canonical timestamped records and quality metadata sent to C3 as A03; offline HMOG records retain participant/session metadata for evaluation, while runtime records stream without ground-truth labels	C / P

Acquisition by modality

For runtime capture, touch and swipe are event types of one touchscreen pointer-event stream and are separated during gesture detection. For offline processing, HMOG raw-touch and gesture files are mapped to the corresponding canonical categories where the available fields support it. Table 2.2 describes the runtime source; the HMOG mapping is verified separately during dataset inspection.

Table 2.2: Acquisition of each behavioral modality.

Modality	Source	Events and attributes	Conditions and limitations	Status
Keystroke	In-app input controls of the prototype (login PIN, transfer fields)	Press and release; `event_time`, `event_type`, `key_slot`	Only while typing. Hold time requires an observable release event (see below). Sparse in navigation and balance enquiry	C / P
Touch	Pointer events delivered to the prototype's views	Down/move/up; `pointer_id`, x, y, `pressure`	Pressure may be unsupported or constant on some devices (U). Only the primary pointer is used in the baseline	C / U
Swipe	Same pointer stream as touch	Same attributes; identified after the fact as a gesture with movement	Occurs mainly in navigation and scrolling; rare during data entry	C / P
Accelerometer	Android sensor framework	`axis_x/y/z`, `event_time`	Continuous while registered. Delivery rate is device dependent; unit and frame to be confirmed (U)	C / U
Gyroscope	Android sensor framework	Same attributes	Not present on every device; capability checked at session start (U)	C / U

Keystroke capture (proposal requiring approval). Hold time needs both a press and a release event. As far as is known, standard Android soft keyboards generally do not report key-release events to an application, so this should be verified against the Android documentation for the target API level before it is relied on. The proposed design therefore uses an in-app keypad whose keys are views of the prototype: a press is the pointer-down on a key and the release is the matching pointer-up, and `key_slot` is assigned from the key position, never from the character. Touches that land on the keypad are recorded as keystroke events, tagged, and excluded from the touch/swipe stream to avoid counting one physical action twice. The alternative, listening to text-change callbacks of a standard keyboard, would provide

press-to-press intervals only and would remove hold time from the feature set. The consequence of the proposed choice is that typing behavior is measured on the prototype keypad rather than on the user's own keyboard (U; the owner of the user-interface module and the project guide to confirm).

Pointer capture. The pointer stream records every down, move and up event of the primary pointer. Because the platform may batch several intermediate positions into one move event, the batched positions are recorded with their own timestamps so that the trajectory resolution is not lost (P). Coordinates are recorded in pixels together with the screen size and density in the device descriptor, so that distances can be converted to physical units in preprocessing (P; this extends the `device_descriptor` field of Part I, Table 1.7, which currently names only model and OS version, and needs approval).

Motion-sensor capture. Accelerometer and gyroscope listeners are registered at session start with a requested sampling period. The requested period is only a hint to the platform; the delivered rate varies between devices and within a session, so the module records both the requested rate and the observed timestamps. The sampling rate is not fixed in the approved documents (U). A rate of a few tens of hertz is a reasonable candidate for pilot evaluation, since hand and posture movements are slow compared with the sensor rate, while a higher rate increases battery use and data volume; the final value should be set from pilot recordings. The raw accelerometer (including the gravity component) is proposed so that static device orientation is not discarded (P).

Timestamps and temporal alignment

Every record carries the platform timestamp of the event itself, not the time at which it was delivered to the application. Pointer events and sensor events may be stamped against different platform clocks (the exact clock of each source is to be verified); the module therefore captures a reference reading of the relevant clocks at session start and converts every native timestamp to session time

$$t = t_{\text{native}} - t_{\text{origin}}, \tag{2.1}$$

where t_{native} is the platform timestamp of a record and t_{origin} is the reading of the same clock at session start, with the offsets between the clocks of different sources measured at that moment. All modalities of one session therefore share one time base, which is the only synchronization mechanism: a sensor sample belongs to the window that contains its timestamp, and no sample-by-sample pairing between a touch event and a sensor reading is attempted. It is assumed that the residual clock offset between sources is small relative to the window length (assumption, to be checked on pilot data). Records of one stream with equal timestamps are ordered by an arrival sequence number (P; an addition to Part I, Table 1.7, requiring approval).

Organization of observations

The logical hierarchy of Part I applies: a participant has sessions, and a session has task segments (login, fund-transfer simulation, balance inquiry, navigation) in which the modalities are recorded concurrently. The acquisition module only delivers records in time-ordered batches annotated with `session_id` and `task_context`. The formation of analysis windows is deliberately left to the Preprocessing Module (stage V5), so that window length, hop and sufficiency rules are defined in one place, as Part I (Table 1.3) assigns segmentation to C3.

Unavailable, interrupted and insufficient observations

Not every device has every sensor, and not every banking screen produces typing or swiping. Table 2.3 lists the situations the module detects and the behavior proposed for each. In none of them does the module invent or interpolate data.

Output and interface to preprocessing

The module produces (i) data records conforming to the logical schema of Part I, Table 1.7, (ii) control records (session start and end, gap markers, task changes) and (iii) a session descriptor containing the device model and OS, screen size and density, sensor presence, requested and observed sensor rate, and pressure availability. These are passed to the Preprocessing Module as A03 and, in collection mode, stored as A15. The Preprocessing Module relies on the guarantees that every record is stamped on the session clock, belongs to one session and stream, and carries no typed content.

Table 2.3: Handling of unavailable or interrupted acquisition (all entries proposed).

Situation	Detection	Behavior of the module
Sensor absent (e.g. no gyroscope)	Capability check at session start	Stream not started; absence recorded in the sensor descriptor; modality unavailable for the whole session
Pressure unsupported or constant	Capability probe, constant-value check in V1	Flag in descriptor; pressure features excluded for that device (feature-schema issue, U)
Application sent to background, screen off, sensor unregistered	Application life-cycle and sensor callbacks	Gap marker with start and end time; no interpolation; windows overlapping the gap are flagged in preprocessing
No typing or swiping for a period	Absence of events	Not treated as a fault; sufficiency is decided in V6
Step-up challenge in progress	Application state	Records tagged `task_context = step-up` and kept out of behavioral windows (P, U); credential entry is never stored
Link to backend interrupted (runtime)	API response failure	Bounded buffering of batches; policy for stale batches TBD (U; coordinate with the API owner)
Non-monotonic or missing timestamps	Checks in V2	Record flagged; handled by the preprocessing rules

Limitations affecting the design

- Keystroke and swipe activity is sparse and task dependent, so some windows will lack one or more modalities. Earlier keystroke work reported the need for long typing blocks per decision (700–900 keystrokes in the work summarized in the Literature Survey), which cannot be assumed for short banking inputs.
- Motion signals depend on posture and activity such as walking, and on grip variation; the prototype collects data in simulated tasks, which may not cover these conditions.
- Sensor characteristics and pressure support vary across Android devices (Part I, Section 1.5.7); cross-device behavior is not established.
- The proposed in-app keypad measures typing on that keypad only.

2.1.2 Preprocessing Module

The Preprocessing Module (component C3, block B05) converts the raw timestamped streams received as A03 into validated, time-aligned and segmented data from which features can be extracted repeatedly during a session (output A04). Part I (Section 1.5.4, Table 1.10) already lists the operations to be applied; this section organizes them into ordered stages, states the input, operation, output and rationale of each, and identifies the parameters that remain open. No stage is applied blindly to every modality, and the stages are designed to flag rather than silently correct data, so that behavioral information is preserved. Preprocessing does not guarantee noise-free data; it limits the influence of invalid, duplicated and implausible records.

Stages

Figure 2.1 (at the end of Section 2.1.3) shows the sequence, and Table 2.4 specifies each stage. Scaling is not performed here: following Part I (Figures 1.7 and 1.9), standardization is applied to feature values after extraction, so no signal-level scaling is proposed apart from unit conversion.

Table 2.4: Stages of the Preprocessing Module (all stages proposed; status in last column).

Stage	Applies to; input	Operation and rationale	Output	Limitations; status
V1	All streams; raw records (A03)	Check required fields for the stream type, allowed event types, finite numeric values, and plausible ranges (coordinates inside the screen, pressure within the device range, sensor values within the sensor's reported range). Corrupt values would otherwise distort timing and statistical features.	Records with `quality_flags`	Range limits depend on the device descriptor and are TBD; P
V2	All streams; flagged records	Reject missing, negative or non-finite timestamps; sort by session time; flag backward jumps and large forward gaps. All timing features depend on correct order.	Ordered streams	Gap tolerance TBD; P
V3	All streams; ordered records	Remove exact duplicates (same stream, time, values and sequence number); keep a count of removed records. Duplicates would create false zero intervals.	De-duplicated streams	Near-duplicates are not removed; P
V4a	Keystroke; press/release events	Pair each press with its release through `key_slot`; flag unmatched presses or releases and exclude them from hold time; compute intervals only between complete consecutive events. Intervals longer than an idle threshold τ_{idle} are capped or excluded from rhythm statistics because a long pause reflects the task, not typing rhythm.	Complete key events	τ_{idle} and cap-versus-exclude rule TBD; P, U
V4b	Touch and swipe; pointer events	Group down–move–up sequences of the primary pointer into gestures; flag cancelled or incomplete gestures; classify a gesture as a tap or a swipe by its displacement and duration (a candidate criterion is the platform's touch-slop distance); route touches on the in-app keypad to the keystroke stream; convert coordinates to physical units with the screen descriptor. Features of gestures require complete gestures.	Gesture records (taps, swipes)	Criterion TBD; multi-touch gestures not used in the baseline; P, U
V4c	Accelerometer, gyroscope; sensor readings	Check delivery intervals against the observed nominal period and insert gap markers where intervals exceed a multiple of it (multiple TBD); align to the session clock; optional smoothing. No resampling is selected (Part I): the baseline window statistics tolerate moderate timing jitter, whereas resampling would only be needed if frequency-domain or sequence inputs were adopted. Gravity is not removed.	Validated, aligned readings	Smoothing method and need TBD; must be causal; P, U
V5	All modalities; prepared data	Segment into time-aligned windows (Section 2.1.2).	Windows	W, S TBD; P, U
V6	Each modality in each window	Count usable events (keystrokes, taps, swipes) or samples (motion coverage) and compare with the minimum for that modality; mark the modality unavailable if below the minimum. Features computed from too few events are unreliable, and imputation would fabricate behavior.	Availability indicator $a_k \in \{0, 1\}$ per modality	Minimum counts TBD; P, U

Continued on next page

Table 2.4: Stages of the Preprocessing Module (continued).

Stage	Applies to; input	Operation and rationale	Output	Limitations; status
V7	Window	Verify internal consistency (start before end, all records inside the window, no window across a session boundary or across a long gap), verify at least one modality is available, and package the window with its metadata.	Preprocessed window (A04) or "no usable window" status	Handling of the latter by C7/C8 is U

Segmentation into analysis windows

Part I names four strategies (event-based, fixed-duration, sliding and sufficiency-based windows) and leaves the choice open. Because keystroke and swipe data are sparse while motion data is dense, a single event count or duration suits no modality alone. The proposal for approval is a *time-aligned container*: sliding windows of duration W and hop $S \leq W$ on the session clock, with each modality segment carrying its own sufficiency indicator (V6). The reasoning is that (i) a fixed hop gives a regular trust-score update interval, which continuous authentication and the latency metric require; (ii) the dense motion stream is naturally described by time windows; and (iii) sparse modalities are handled by the indicator rather than by irregular event-based windows. Window n covers

$$[\, t_n^s,\, t_n^e \,) = [\, t_0 + nS,\; t_0 + nS + W \,), \tag{2.2}$$

where t_0 is the session origin, W is the window duration and S is the hop. Values of W and S are not established; they are to be chosen from a small candidate set on validation data, and they determine the update interval defined in Part I, Section 1.4.2. Partition assignment is made before windowing and no window crosses a session boundary (Part I, Section 1.5.5), because overlapping windows are near-duplicates. At runtime a window is closed a short grace delay δ_g after t_n^e so that late-delivered samples with earlier timestamps are included; δ_g adds to authentication latency and is to be measured (TBD).

Parameters requiring confirmation

Table 2.5 lists parameters that this section depends on but that the approved documents do not fix. None is represented as a project decision.

Table 2.5: Preprocessing parameters that are not yet established.

Parameter	Symbol	Proposed way to fix it
Window duration and hop	W, S	Compare a small candidate set on the validation partition; must agree with the update interval of Part I
Grace delay	δ_g	Measure sensor delivery delay in pilot sessions
Idle threshold for typing	τ_{idle}	Inspect interval distributions of pilot data; decide cap or exclusion
Minimum usable events or samples	$n_{\min}^{(k)}$	Per modality; large enough that all window statistics are defined (at least two intervals for a standard deviation)
Sensor request rate and gap multiple	—	Pilot recordings on the target devices
Tap/swipe criterion	—	Platform touch-slop distance and a maximum tap duration, verified on pilot data
Smoothing filter (if any)	—	Introduce only if pilot data show a benefit; must be causal

Offline and runtime consistency

The same stage definitions (V1–V7) are used in both contexts (A18 in Figure 1.3). Offline, the stages read HMOG sessions after conversion to the canonical schema; at runtime they receive batches incrementally. To avoid a mismatch between training and runtime, every operation is required to be *causal*: it may use only records up to the window end plus the grace delay. A global operation that looks ahead in a stored session, such as a non-causal smoothing filter, is not permitted unless it can also be run in real time.

2.1.3 Feature Extraction Module

The Feature Extraction Module (component C4, block B06) converts each preprocessed window (A04) into a fixed-length numerical descriptor per available modality (A05). The statistics follow the candidate feature groups of Part I (Table 1.9 and Figure 1.8); this section defines them precisely, states their source attributes and dependencies, and separates features that depend on unconfirmed capabilities. The catalogue is a candidate set: the final list and the dimension d are fixed by feature selection (Part I, Section 1.6.2) and are not decided here. Statistical timing and motion descriptors of this kind are used in the surveyed work (separate keystroke and swipe feature sets, and feature-level fusion of touch, keystroke and accelerometer features); that work justifies the type of feature, not the particular choices below.

Notation and general rules

Let a window contain, for modality $k \in \{\text{key}, \text{touch}, \text{swipe}, \text{motion}\}$, a set of events or samples. For a set of values $\{z_i\}_{i=1}^{n}$ the statistics used are

$$\mu = \frac{1}{n}\sum_{i=1}^{n} z_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(z_i - \mu)^2}, \tag{2.3}$$

together with the median, minimum and maximum, matching the population form used in Part I. A modality contributes features only if it is marked available in V6, so that every statistic below is defined. Counts n_k are carried as metadata (support), not as features, until the team decides whether activity level is behaviorally meaningful or merely task dependent (U).

Keystroke features

Let a window contain n complete key events ordered by press time, with press time t_i^p and release time t_i^r in session time. The hold time, flight time and press-to-press interval are

$$\text{HT}_i = t_i^r - t_i^p, \tag{2.4}$$

$$\text{FT}_i = t_{i+1}^p - t_i^r, \tag{2.5}$$

$$\text{PP}_i = t_{i+1}^p - t_i^p. \tag{2.6}$$

Flight time can be negative when the next key is pressed before the previous one is released; the sign is kept, because overlap is itself a feature of typing style, and intervals above τ_{idle} are handled as in V4a. Typing speed over a window is $\kappa = (n-1)/(t_n^p - t_1^p)$ for $n \geq 2$. Part I leaves the flight-time definition open (release-to-press or press-to-press); FT is used as the baseline, as in Part I, Figure 1.7, and PP is optional because it remains computable when a release is unreliable. Because `key_slot` is not a character, digraph-specific features (timing per key pair) cannot be formed; all keystroke features are key-agnostic summaries.

Table 2.6: Candidate keystroke features.

ID	Feature	Source	Relevance and limits	Status
K1	μ, σ, median, min, max of HT	press, release time	How long keys are held; needs observable release (Section 2.1.1)	C / P
K2	μ, σ, median, min, max of FT	release, next press	Typing rhythm; definition open in Part I	C / U
K3	Typing speed κ	press times	Overall pace; diluted by pauses	P
K4	Same five statistics of PP (optional)	press times	Rhythm without releases; adds five features, so inclusion needs approval	P

Touch features

A tap g has down time t_g^d, up time t_g^u, down position (x_g, y_g) in physical units (or normalized by screen size) and mean pressure $\bar{p}_g$ over its samples. Per tap, duration is $D_g = t_g^u - t_g^d$, and the interval between consecutive taps is $I_g = t_{g+1}^d - t_g^d$. Window features are the mean and standard deviation of D, I and $\bar{p}$, and the standard deviations σ_x, σ_y of the down positions, which describe how widely the touches are spread. The mean position is not included, because it mainly reflects the layout of the screen in use rather than the user.

Table 2.7: Candidate touch features.

ID	Feature	Source	Relevance and limits	Status
T1	μ, σ of tap duration D	down, up time	Deliberateness of a tap	C
T2	μ, σ of inter-tap interval I	down times	Pace of interaction; task dependent	P
T3	μ, σ of mean pressure $\bar{p}$	pressure samples	Touch force; availability unconfirmed, may be constant on some devices	C / U
T4	σ_x, σ_y of down positions	x, y at down	Spread of touch locations; depends on screen layout and task	C / P

Swipe features

A swipe has samples (t_j, x_j, y_j), $j = 0, \ldots, J$, in physical units. Its duration, path length, displacement, mean speed, straightness and direction are

$$T = t_J - t_0, \qquad L = \sum_{j=1}^{J} \|\mathbf{p}_j - \mathbf{p}_{j-1}\|, \qquad \Delta = \|\mathbf{p}_J - \mathbf{p}_0\|, \tag{2.7}$$

$$\bar{v} = \frac{L}{T}, \qquad \rho = \frac{\Delta}{L}\ (L > 0), \qquad \theta = \operatorname{atan2}(y_J - y_0,\ x_J - x_0), \tag{2.8}$$

where $\mathbf{p}_j = (x_j, y_j)$. The mean speed $\bar{v}$ is the quantity v = path length/duration of Part I. Peak speed is $v_{\max} = \max_j \|\mathbf{p}_j - \mathbf{p}_{j-1}\|/(t_j - t_{j-1})$, computed only over steps whose time difference exceeds a small minimum, because very short steps give unstable speeds. Since an angle is circular, direction is represented by $(\cos\theta, \sin\theta)$ and averaged as a vector. Window features are the mean and standard deviation of T and $\bar{v}$, and the mean of L, Δ, ρ, $v_{\max}$, $\bar{p}$, $\cos\theta$ and $\sin\theta$ over the swipes in the window. Full trajectory descriptors (Part I, Table 1.9, "trajectory summary") are reduced to the straightness ratio ρ to limit the feature count.

Table 2.8: Candidate swipe features.

ID	Feature	Source	Relevance and limits	Status
S1	μ, σ of duration T	times	Speed and deliberateness of a swipe	C
S2	μ, σ of mean speed $\bar{v}$	path, time	Velocity profile summary	C
S3	Mean path length L, displacement Δ	x, y	Extent of the gesture; depends on screen size (physical units)	C
S4	Mean straightness ρ	x, y	Path shape; reduces to 1 for straight swipes	P
S5	Mean peak speed $v_{\max}$	x, y, time	Sensitive to timing resolution of batched samples	C / P
S6	Mean $(\cos\theta, \sin\theta)$	x, y	Direction habit; task dependent (scroll direction)	P
S7	Mean pressure $\bar{p}$	pressure	Availability unconfirmed	C / U

Accelerometer and gyroscope features

For each sensor $s \in \{\text{acc}, \text{gyro}\}$ and axis $a \in \{x, y, z\}$, let $\{s_a[k]\}_{k=1}^{N}$ be the samples whose timestamps fall in the window. The features are the mean, standard deviation, minimum and maximum of $s_a[k]$ and the signal energy

$$E_{s,a} = \frac{1}{N} \sum_{k=1}^{N} s_a[k]^2, \tag{2.9}$$

defined as a mean square so that it does not grow with the number of samples, which varies with the delivered sampling rate. Optionally the magnitude $m_s[k] = \sqrt{s_x[k]^2 + s_y[k]^2 + s_z[k]^2}$ is summarized by its mean and standard deviation, which describes overall movement intensity independently of device orientation. With the raw accelerometer, the axis means mainly reflect static orientation (posture), which is informative but also changes with activity. The evidence that more sensors are not always better motivates keeping the accelerometer and gyroscope groups separable so that their contribution can be tested by ablation. Frequency-domain features are not included, because they would require uniform sampling (see V4c) and are outside the approved scope.

Table 2.9: Candidate motion features (per sensor).

ID	Feature	Source	Relevance and limits	Status
M1	Per-axis mean	axis values	Posture or typical orientation	P
M2	Per-axis σ, min, max	axis values	Movement variability; min and max are sensitive to outliers	P
M3	Per-axis energy $E_{s,a}$	axis values	Movement intensity	P
M4	Mean and σ of magnitude (optional)	axis values	Orientation-independent intensity	P

Features by dependency on unconfirmed capabilities

Table 2.10 separates the catalogue by what it depends on, as required for team approval. Group A depends only on attributes already in the logical schema; Group B depends on attributes whose availability is unconfirmed; Group C needs capabilities that are not part of the schema or the approved scope.

Table 2.10: Candidate features grouped by dependency.

Group	Features	Dependency and decision needed
A	K1–K3, T1, T2, T4, S1–S6, M1–M3 (with M4 optional)	Time, coordinates, key-slot and axis values in Part I, Table 1.7; gyroscope features need the gyroscope to be present
B	T3, S7 (pressure); K4; S5 and S6 resolution; gyroscope block	Pressure support per device; optional PP; sensor presence; confirm on target devices
C	Contact-area features; per-key (digraph) timing; frequency-domain motion features; session-level context features	Contact area is not in the schema; digraph timing conflicts with the no-characters rule; frequency features need resampling; all require approval

Proposed initial set. For the first implementation it is proposed to use Group A, add the pressure features only if pressure is confirmed to vary on the target devices, and decide M4, K4 and the min/max statistics by feature selection. The full catalogue above has 11 keystroke (K1–K3), 8 touch, 11 swipe and 34 motion features (including the optional magnitude features M4 and excluding the optional K4), 64 in total before selection. This number is the size of the candidate catalogue, not the final dimension d. The motion block is more than half of this catalogue; because the Isolation Forest chooses features at random and the Siamese and Autoencoder models are sensitive to feature scale and block size, block balance should be reviewed together with the model owner (U).

Feature record, ordering and missing values

The module outputs one *feature record* per window, with the fields in Table 2.11. Feature order within each modality vector is fixed by a versioned feature schema stored with the shared definitions (element K3 of Figure 1.3), so that training and runtime vectors line up. The record contains only the vectors of available modalities; it never contains imputed values, because the choice between a neutral value with an availability flag and exclusion of the window is a team decision assigned to the Fusion Layer (Part I, Figure 1.9). Within an available modality all features are defined by construction (V6). A feature group that cannot exist on a device (for example pressure on a device without it) is excluded from the schema for all devices used in one experiment, so that the dimension does not vary between training and runtime.

Table 2.11: Logical structure of the feature record (A05).

Field	Content
`sample_id`, `session_id`	Identifier of the window and of its session; `participant_id` offline only
`window_start`, `window_end`	Boundaries on the session clock
$\mathbf{x}_{\text{key}}, \mathbf{x}_{\text{touch}}, \mathbf{x}_{\text{swipe}}, \mathbf{x}_{\text{motion}}$	Raw (unstandardized) modality vectors, present only if available
$a_{\text{key}}, a_{\text{touch}}, a_{\text{swipe}}, a_{\text{motion}}$	Availability indicators
n_k	Support counts per modality
`schema_version`	Version of the feature order and definitions
`quality_flags`	Aggregated flags from preprocessing

Standardization and model input

Each feature is standardized as in Part I, $x'_j = (x_j - \mu_j)/\max(\sigma_j, \varepsilon)$, where μ_j and σ_j are fitted on the training partition only, stored, and reused unchanged for validation, test, enrollment and runtime data, and ε is a small constant guarding against zero variance. Part I shows this step between extraction and fusion without naming its owner; it is described here for completeness, and whether it belongs to C4 or to the Fusion Layer (C5) is unresolved. The standardized modality vectors are passed to the Fusion Layer, which concatenates them in a fixed order. The Siamese Neural Network, the Autoencoder and the Isolation Forest of Part I then receive the same fused vector, and the One-Class SVM baseline can use it as well. This module does not select, tune or change any model.

Interfaces and processing allocation of the three modules

Table 2.12 gives the logical interfaces among the three data-processing modules and the adjacent components; arrow identifiers refer to Part I, Table 1.4. Figure 2.1 presents the complete sequence.

Table 2.12: Logical interfaces of the data-processing modules.

Arrow	From → To	Content
A02	Application → C2	Live touch, swipe, key and device-motion events
A03	C2 → C3	Timestamped raw records, control records and session descriptor
A04	C3 → C4	Validated windows with per-modality segments and availability indicators, or "no usable window" status
A05	C4 → C5	Feature records (vectors of available modalities, indicators, counts, schema version)
A15	C2 → C12	Labeled raw sessions in collection mode
A18	C3–C5 ↔ C13	Shared preprocessing, feature and fusion definitions

Proposed allocation between device and backend. Part I states that the split between on-device and backend processing is not yet decided. Table 2.13 gives a recommendation for the prototype, based on latency, bandwidth, privacy and implementation effort. It is not an approved decision.

Table 2.13: Recommended allocation of the data-processing operations (requires approval).

Operation	Location	Justification	Dependency
Capture and timestamping	Mobile device	Sensors and input events exist only on the device	Application and sensor access
Basic validation (V1–V3)	Mobile device	Inexpensive; avoids transmitting corrupt records	Descriptor for ranges
Pairing, gesture grouping, windowing, V4–V7	Backend (prototype)	One Python implementation shared with offline training limits mismatch (K3); on-device ports increase effort	Raw-record transmission
Feature extraction and standardization	Backend (prototype); on-device optional later	Same reason; on-device extraction would reduce bandwidth and keep raw timing on the device but needs a second implementation	Decision on payload and on C4/C5 ownership
Transmission (REST API)	Device to backend	Payload is raw window batches (backend option) or feature records (device option); size depends on W, S and the sensor rate (TBD)	Final deployment arrangement
Offline dataset storage	Workstation / controlled project storage	Use HMOG files under the dataset terms; do not redistribute the source dataset	License, access and retention rules

The recommendation trades bandwidth and the exposure of raw timing data against consistency and simplicity. It must be agreed with the project guide and with the owner of the authentication module, because it affects latency and the API payload. HMOG is an offline training/evaluation source; it is not the runtime API payload and must not be copied into the deployed application.

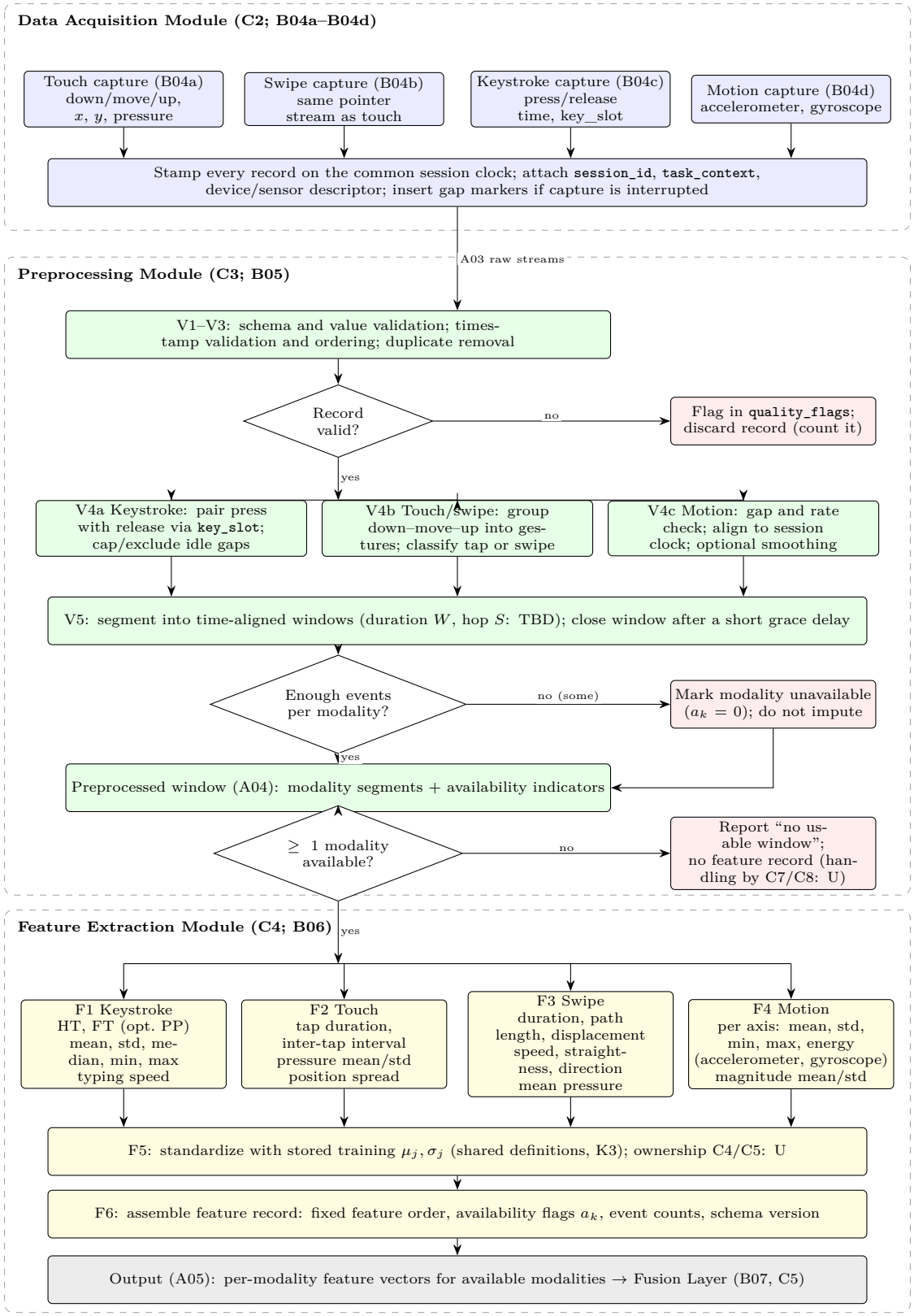


Figure 2.1: Data acquisition, preprocessing and feature extraction pipeline of the proposed system. Blue blocks hold raw timestamped observations, green blocks preprocessed data, yellow blocks extracted features; red blocks show how invalid or insufficient observations are handled. Identifiers B04–B07, C2–C5, A03–A05 and K3 refer to Figure 1.3 and Tables 1.2–1.4 of Part I. Window parameters and the ownership of standardization are unresolved (U).

2.1.4 Machine Learning Model Module

Purpose and position

The Machine Learning Model Module is part of the Authentication layer (components B08–B10, C6 in Part I). For each window, it takes the fused behavioral vector $x_t \in \mathbb{R}^d$ and the claimed user's enrolled profile. It returns three scores: a similarity score, a reconstruction error and an anomaly score. These scores are evidence, not decisions. The Prediction and Decision Module (Section 2.1.5) converts them into a trust score and a session action.

Table 2.14: Machine Learning Model Module: input, processing, output and interfaces.

Item	Description
Input	Fused, normalized feature vector x_t (from Fusion Layer B07); enrolled user profile and trained model artifacts (from Profile and Model Store B14)
Processing	Run the SNN, AE and IF on the same x_t
Output	s_{sim}, s_{ae} and s_{if} for window t, plus the One-Class SVM score in evaluation mode only
Interfaces	Receives data from C5 (Fusion) and C11 (Profile/Model Store); sends scores to C7 (Trust Score Generator)

Dataset used by the models

The system is developed and evaluated on the public HMOG dataset (Hand Movement, Orientation, and Grasp), which records keystroke timing, touch and swipe events, and accelerometer and gyroscope readings from the same participants over multiple sessions. Model development and evaluation use HMOG only; no volunteer data collection is planned for training or evaluation. The collection mode of the prototype (Part I, A15; Section 2.5) is limited to optional pilot and demonstration sessions, subject to approval. HMOG is used under its license terms and cited in the report.

Table 2.15: Use of HMOG data in each partition.

Partition	Objective A (session-level)	Objective B (participant-level)
Training	Training sessions of all participants: scaler fitting, SNN pairs, genuine data for AE/IF	All sessions of training participants only
Enrollment	Enrollment sessions of each participant (references only)	Early sessions of validation/test participants (references only)
Validation	Validation-probe sessions: model selection, score normalization, weights, α, τ_1, τ_2	Validation participants
Test	Test-probe sessions, later than enrollment; used once	Later sessions of test participants; used once

Processing rules.

1. **Partition before windowing:** sessions (Objective A) or participants (Objective B) are assigned to partitions before windows are created, so overlapping windows never cross a split.
2. **Own feature extraction:** features are re-extracted from the raw HMOG events using the feature definitions of Part I (hold and flight time, swipe velocity, duration, pressure, motion statistics). Values in the dataset are not edited.
3. **Fixed feature schema:** the fused vector x_t has a fixed length d. A modality with insufficient data in a window is marked unavailable by the preprocessing and feature modules (Sections 2.1.2 and 2.1.3); the Fusion Layer then applies a neutral value with an availability flag (Option A in Part I, to be confirmed). Features that HMOG does not record, such as pressure if unavailable, are excluded from the schema.
4. **Scaling without leakage:** scaler parameters are fitted on training data only and reused for all other data.

Limitations. HMOG tasks are reading, writing and map navigation, not banking tasks, so the evaluation does not measure behavior in login, transfer or balance-inquiry screens. The device models and sensor sampling rates are those used in the dataset. The mobile prototype demonstrates the pipeline, but a live demonstration depends on the app producing features comparable to HMOG's.

Models

Siamese Neural Network (SNN): similarity learning. Two weight-shared encoder branches map feature vectors into an embedding space:

$$z = f_\theta(x) \in \mathbb{R}^k.$$

The network is trained on HMOG training participants using pairs of samples, where positive pairs belong to the same user and negative pairs belong to different users. Contrastive loss or triplet loss may be used. Pairs are formed within one partition only.

At runtime, x_t is embedded and compared with the user's reference embeddings r_u, which are constructed from enrollment samples:

$$s_{\text{sim}}(x_t) = \text{sim}\big(f_\theta(x_t), r_u\big).$$

The similarity function may be defined as $1/(1+D)$ or cosine similarity, where r_u represents the mean or maximum similarity over the reference set.

The SNN determines whether the observed behavior resembles that of the enrolled user. A higher similarity score indicates more genuine behavior. A new user can be enrolled using a few samples without retraining the shared encoder.

Autoencoder (AE): reconstruction-based one-class model. The encoder h_ψ compresses the input vector into a latent representation $z \in \mathbb{R}^m$, where $m < d$. The decoder g_ϕ reconstructs the input as $\hat{x}$. The AE is trained only on the legitimate user's normal behavior using mean squared error (MSE) loss. Its score is

$$s_{\text{ae}}(x_t) = \big\| x_t - g_\phi\big(h_\psi(x_t)\big) \big\|^2.$$

The reconstruction error indicates how well the observed behavior fits the learned structure of normal behavior. A higher error indicates more anomalous behavior. The AE can capture nonlinear correlations across modalities.

Isolation Forest (IF): tree-based outlier detection. An ensemble of T random isolation trees is fitted to the genuine user's feature vectors. The anomaly score is

$$s_{\text{if}}(x_t) = 2^{-\frac{\mathbb{E}[h(x_t)]}{c(n)}}.$$

Here, $\mathbb{E}[h(x_t)]$ denotes the expected path length for isolating the sample, and $c(n)$ is the normalization factor for a sample size of n. The IF determines how rare the observed behavior is within the user's feature distribution. A score close to 1 indicates anomalous behavior. Its inference cost is low because it requires only tree traversal.

One-Class SVM (OC-SVM): comparison baseline only. The OC-SVM is fitted using genuine feature vectors only and an RBF kernel. It learns a boundary around normal behavior in feature space. Its anomaly score is defined as

$$s_{\text{ocsvm}}(x_t) = -f(x_t),$$

where $f(x_t)$ is the signed distance to the learned boundary. A higher score indicates more anomalous behavior. The OC-SVM uses the same input vector, data partitions and preprocessing as the proposed models. It is not included in the trust-score calculation. Instead, it is evaluated alongside the other models using EER, FAR, FRR, F1-score and latency as a standard comparison baseline (Part I, Section 1.6.7).

How the models contribute to continuous authentication

Table 2.16: Roles of the models in continuous authentication.

Model	View of behavior	Catches
SNN	Closeness to enrolled examples	An impostor whose behavior is clearly different from that of the enrolled user
AE	Fit to the learned structure of normal behavior	Unusual feature combinations that violate learned correlations
IF	Rarity in feature space	Extreme values and outliers at low computational cost

A deviation underestimated by one model may be detected by another, which is why their scores are combined. The contribution of each model and of the combined system will be established through comparative evaluation; no results are available yet. Each new window passes through the same three models, allowing identity to be reassessed throughout the session rather than only at login.

Training and enrollment summary

The SNN is a shared model trained on HMOG training participants (Algorithm 1). The AE and IF are fitted per user on genuine enrollment and training sessions only; whether they are per-user or global remains open in Part I (Section 1.5.5), and per-user fitting is assumed in Algorithm 1. Impostor data is never used to fit the AE, IF or OC-SVM, only as probes for tuning and evaluation. All hyperparameters (Part I, Table 1.15) are tuned on validation data.

2.1.5 Prediction and Decision Module

Purpose

This module covers the Trust Score Generator (B12, C7) and the Adaptive Authentication Engine (B13, C8), with the Step-Up Handler (B02, C9) on the application side. It converts model evidence into a session action.

Table 2.17: Prediction and Decision Module: inputs, processing, outputs and interfaces.

Item	Description
Input	s_{sim}, s_{ae}, s_{if}; score-normalization statistics; fusion weights; α; τ_1, τ_2; step-up result
Processing	Normalize, orient, fuse, smooth, threshold
Output	Trust score T_t, smoothed score $\widetilde{T}_t$, session action (continue / step-up / terminate)
Interfaces	Receives from C6; sends action to C1 (application) and step-up request to C9

Trust score generation

Step 1: score normalization. Each model score is normalized to the range $[0, 1]$ using statistics from genuine validation data. Min–max normalization with clipping or percentile-rank normalization can be used.

$$\text{norm}(s) = \text{clip}\left(\frac{s - s_{\min}}{s_{\max} - s_{\min}}, 0, 1\right) \tag{2.10}$$

Step 2: score orientation. The scores are oriented so that a higher value always indicates behavior that is more consistent with the genuine user, where $\text{norm}(\cdot)$ is Equation (2.10) applied with the statistics of the respective model:

$$\hat{s}_{\text{sim}} = \text{norm}(s_{\text{sim}}), \tag{2.11}$$

$$\hat{s}_{\text{ae}} = 1 - \text{norm}(s_{\text{ae}}), \tag{2.12}$$

$$\hat{s}_{\text{if}} = 1 - \text{norm}(s_{\text{if}}). \tag{2.13}$$

Step 3: score fusion. The normalized scores are combined using a weighted sum. The weights are tuned using validation data.

$$T_t = w_{\text{sim}}\hat{s}_{\text{sim}} + w_{\text{ae}}\hat{s}_{\text{ae}} + w_{\text{if}}\hat{s}_{\text{if}} \tag{2.14}$$

The fusion weights satisfy the constraint

$$w_{\text{sim}} + w_{\text{ae}} + w_{\text{if}} = 1 \tag{2.15}$$

Step 4: exponential moving average. The trust score is smoothed using an exponential moving average so that a single noisy window does not trigger an alert.

$$\widetilde{T}_t = \alpha T_t + (1 - \alpha)\widetilde{T}_{t-1} \tag{2.16}$$

A smaller value of α provides greater stability but results in a slower response to an impostor. This trade-off is tuned using the False Acceptance Rate (FAR), False Rejection Rate (FRR) and detection latency. The initial value $\widetilde{T}_0$ at the start of a session is TBD.

Calibration data. Normalization statistics, fusion weights, α, τ_1 and τ_2 are calibrated using HMOG validation data only. Test data is used once for final reporting.

Interpretation of the score. The smoothed trust score $\widetilde{T}_t \in [0, 1]$ represents the system's running confidence that the person operating the device is the enrolled user. It is not a probability. Its meaning is determined by calibration on validation data.

Decision rule

Two thresholds, τ_1 and τ_2, are applied to the smoothed trust score, where $\tau_1 > \tau_2$. After a successful step-up, a grace counter suppresses further challenges for N windows (Algorithm 4); N is TBD.

Table 2.18: Trust-score-based authentication decision rules.

Condition	Action	Meaning
$\widetilde{T}_t \geq \tau_1$	Continue	Behavior consistent with the legitimate user
$\tau_2 \leq \widetilde{T}_t < \tau_1$	Step-up authentication	Behavior is suspicious, so the user is challenged
$\widetilde{T}_t < \tau_2$	Terminate	Severe deviation

Step-up authentication outcomes

- **Success:** the session continues.
- **Failure:** the session is terminated.

2.1.6 Database Module

Purpose

The Database Module holds everything the system must persist: users and devices, sessions, windows and feature vectors, enrollment profiles, trained model artifacts and preprocessing parameters, runtime scores, trust values, decisions and evaluation results. It serves both paths in Part I:

- **Offline path:** Dataset Store (B15), Training and Enrollment (B16) and Evaluation (B17).
- **Runtime path:** Profile and Model Store (B14) and authentication records.

The logical design of the stored entities is given in Section 2.4.

Table 2.19: Database Module: input, processing, output and interfaces.

Item	Description
Input	Imported HMOG sessions, extracted features, trained artifacts, runtime scores, trust values, decisions and evaluation metrics
Processing	Store, version and retrieve records while enforcing referential integrity
Output	Training and test data, user profiles and models for inference, and an audit trail of decisions
Interfaces	C12, C11, C13, C14, C6, C7, C8 and the REST API (C10)

Design principles

The database design follows Part I, Section 1.5.7:

- **Privacy:** participants are identified only by pseudonymous codes. No typed characters, PINs, passwords or banking data are stored; keystroke data is restricted to timing information.
- **Dataset use:** the HMOG records keep their citation and license information and are used according to the applicable terms. Users imported from HMOG are marked by an origin field so they remain separate from any live demonstration users.
- **Storage:** high-volume raw streams are kept as restricted files and referenced through `raw_data_ref`; features and results are stored in tables. Model files and scaler parameters are stored as versioned artifacts.
- **Technology:** PostgreSQL or Firebase may be used. The logical design is independent of the final choice.

2.1.7 User Interface Module

Purpose

The User Interface Module corresponds to the Mobile Banking Application Prototype (C1, B01) and the Secondary Authentication Handler (C9, B02) of the architecture in Part I, Section 1.4. It is the only part of the system the user interacts with directly. It provides the banking-style screens in which behavioral data is generated, and it applies the session actions returned by the Decision layer. It does not perform feature extraction, model inference, trust-score computation or threshold evaluation; these belong to the Data Processing, Authentication and Decision layers. All elements described here are **proposed design**; none has been implemented or experimentally verified.

Functional responsibilities

Table 2.20 lists the responsibilities of the module.

Table 2.20: Responsibilities of the User Interface Module (proposed).

Function	Description	Related component
Initial login	Hosts the entry point where the user completes the existing PIN or fingerprint login. A successful login starts the monitored session (A01).	B00, C1
Banking task screens	Provides the simulated banking interactions defined in Part I: login, fund-transfer simulation, balance inquiry and navigation. Numeric entry uses the proposed in-app keypad (Section 2.1.1; U).	C1
Interaction event forwarding	Passes live touch, swipe, keystroke and device-motion events to the Data Acquisition Module without requiring any deliberate user action (A02).	C1, C2
Session state handling	Maintains the state of the active session (started, continuing, awaiting step-up, terminated).	C1
Session action application	Applies the action returned by the Adaptive Authentication Engine: continue, request step-up authentication or terminate (A09, A10).	C1, C8
Step-up challenge presentation	Presents the secondary authentication challenge when requested and returns its success or failure result (A11). The challenge mechanism is **TBD**.	C9, C8
Application/backend communication	Exchanges requests and responses with the proposed REST API Interface (A12). The payload depends on the final deployment arrangement (**TBD**).	C10

Inputs and outputs

Table 2.21 summarizes the data entering and leaving the module. Arrow identifiers refer to Part I, Table 1.4.

Interaction with the authentication system

The interaction follows the runtime flow of Part I, Section 1.4.2.

1. After the initial login succeeds, the module starts the monitored session and begins forwarding interaction events to the Data Acquisition Module.
2. Monitoring runs passively in the background. The module shows no authentication prompt or trust indicator during normal use. The trust score is an internal intermediate value and is not displayed to the user.
3. The module receives the session action from the Decision layer and applies it:
   - **Continue:** the session proceeds without interruption and monitoring continues with the next behavioral window.
   - **Step-up:** the module presents the secondary authentication challenge through the Secondary Authentication Handler and reports the result. If it succeeds, the session continues. If it fails, the session is terminated.
   - **Terminate:** the module ends the session and returns the user to the login state.

Table 2.21: Inputs and outputs of the User Interface Module.

	Item	Source / destination
Inputs	User touch, swipe and typing interactions; PIN or fingerprint credential at login	User, mobile device
	Successful-login event (A01)	Initial Login (B00)
	Continue or terminate action (A09)	Adaptive Authentication Engine (B13)
	Step-up request (A10)	Adaptive Authentication Engine (B13)
Outputs	Live interaction events (A02)	Data Acquisition (B04a–d)
	Step-up success or failure (A11)	Adaptive Authentication Engine (B13)
	Application requests (A12)	REST API Interface (B03)
	Banking-task screens, step-up challenge and session outcome shown to the user	User

4. Communication with the backend machine learning engine passes through the proposed REST API Interface. Whether inference runs on the device or on the backend is **unresolved** and does not change the responsibilities of this module.

Design considerations

- **Passive operation:** behavioral capture adds no extra steps for the legitimate user, consistent with the overall objective in Part I, Section 1.2.
- **Data minimization:** behavioral data collection is limited to the approved fields listed in Part I, Table 1.7. Typed characters, PINs, passwords and sensitive banking inputs are not retained as behavioral records. Banking tasks use a simulated application and do not involve real accounts or transactions (Part I, Section 1.5.7).
- **Separation of concerns:** thresholds, fusion weights and model outputs are handled outside the interface, so the interface does not depend on the choice among the Siamese Neural Network, Isolation Forest and Autoencoder.
- **Collection mode:** the same prototype is proposed for optional pilot and demonstration sessions (Part I, Section 1.5.1). Its consent and session-labeling screens are covered in Section 2.5.

Status and open items

The module is a proposed design. The following items remain **TBD**: the step-up authentication mechanism, the form of the messages shown on step-up or termination, the application/API payload, and the split between on-device and backend processing. These will be fixed during implementation and are not assumed here.

2.2 Functional Design

The functional design describes how the proposed continuous authentication system behaves from the viewpoint of its user, its control flow, its component interactions and its logical software structure. The four UML diagrams in this section are consistent with the architecture and data flow of Design Document Part I (Sections 1.3 to 1.6) and do not repeat it. The diagrams do not fix numerical thresholds, fusion weights, window parameters, the device/backend processing split or the step-up mechanism; the proposed trust-score method is given in Section 2.1.5. Table 2.22 lists the diagrams.

Design basis. The diagrams are validated against Part I as follows.

- **Actor:** the legitimate banking user is the only external human actor. The initial PIN/fingerprint mechanism and device sensors are external elements but not actors. No administrator, police, payment gateway or third-party identity provider is introduced.

Table 2.22: Diagrams of the functional design.

Section	Diagram	Aspect explained
2.2.1	Use case diagram	Externally visible functions; mandatory versus conditional behavior
2.2.2	Activity diagram	End-to-end control flow and decision branches
2.2.3	Sequence diagram	Chronological message exchange among components
2.2.4	Class diagram	Proposed logical software structure

- **Modalities:** keystroke dynamics, touch interactions, swipe gestures, accelerometer and gyroscope readings (four streams, as in Part I, Figure 1.3).
- **Models:** Siamese Neural Network (similarity score), Isolation Forest (anomaly score) and Autoencoder (reconstruction error). The One-Class SVM is the comparative baseline only.
- **Outcomes:** continue the session, request step-up (secondary) authentication, or terminate the session. A failed step-up and a severe deviation both terminate the session.
- **Not fixed by the diagrams:** thresholds, trust-score weights, window parameters, fusion strategy, device/backend split and step-up mechanism.
- **Status:** all figures show a proposed design, not an implemented system.

2.2.1 Use Case Diagram

The use case diagram identifies the externally visible functions of the system and separates mandatory behavior from conditional behavior. The legitimate banking user is the only external actor; all processing stages are internal to the system boundary.

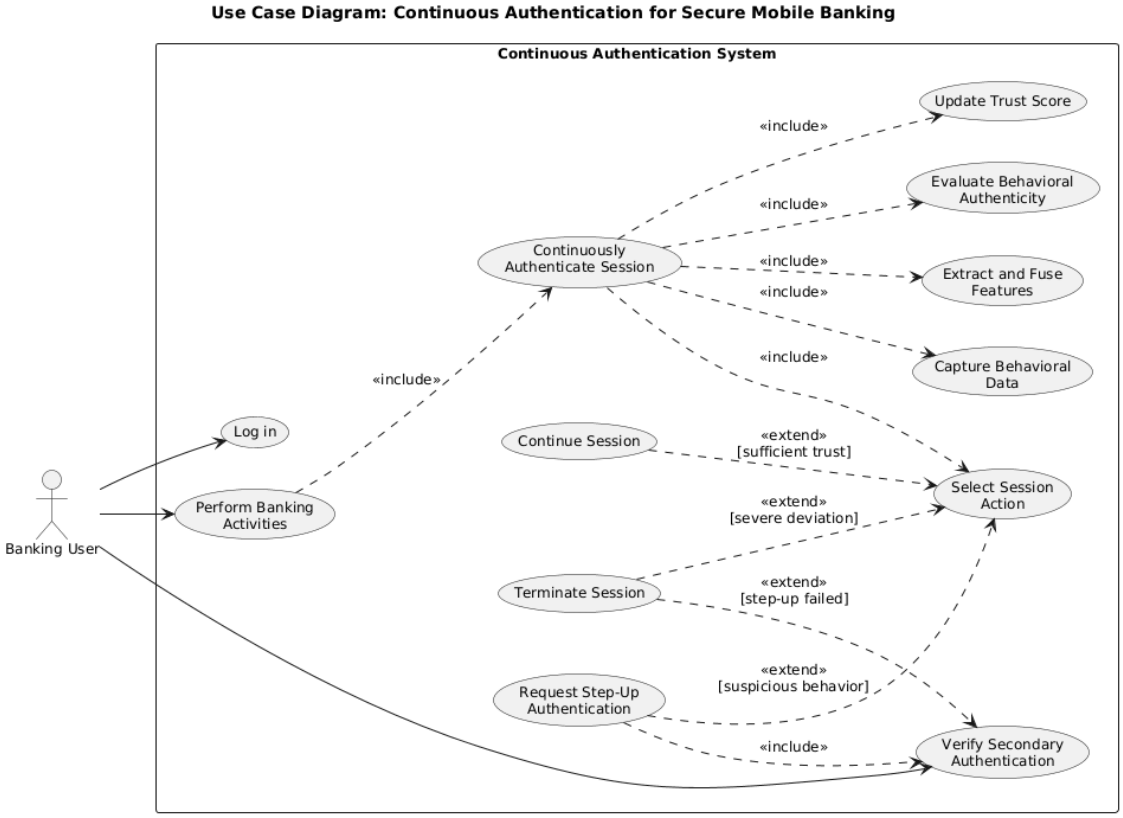


Figure 2.2: Use case diagram of the proposed behavioral biometric-based continuous authentication system for secure mobile banking.

The user logs in with PIN or fingerprint and performs banking activities. Performing banking activities includes continuous authentication of the session, which in turn includes capturing behavioral data, extracting and fusing behavioral features, evaluating behavioral authenticity, updating the trust score and selecting a session action. These inclusions are mandatory. The three session outcomes are modeled as extensions of the action-selection use case: continuing the session when trust is sufficient, requesting step-

up authentication when behavior is suspicious, and terminating the session when deviation is severe. Step-up authentication includes verification of the secondary authentication, in which the user participates, and a failed verification extends to session termination. Figure 2.2 supports design objectives DO1 to DO4 and DO6 of Part I.

2.2.2 Activity Diagram

The activity diagram shows the end-to-end control flow of one authentication cycle and the decision branches that follow it.

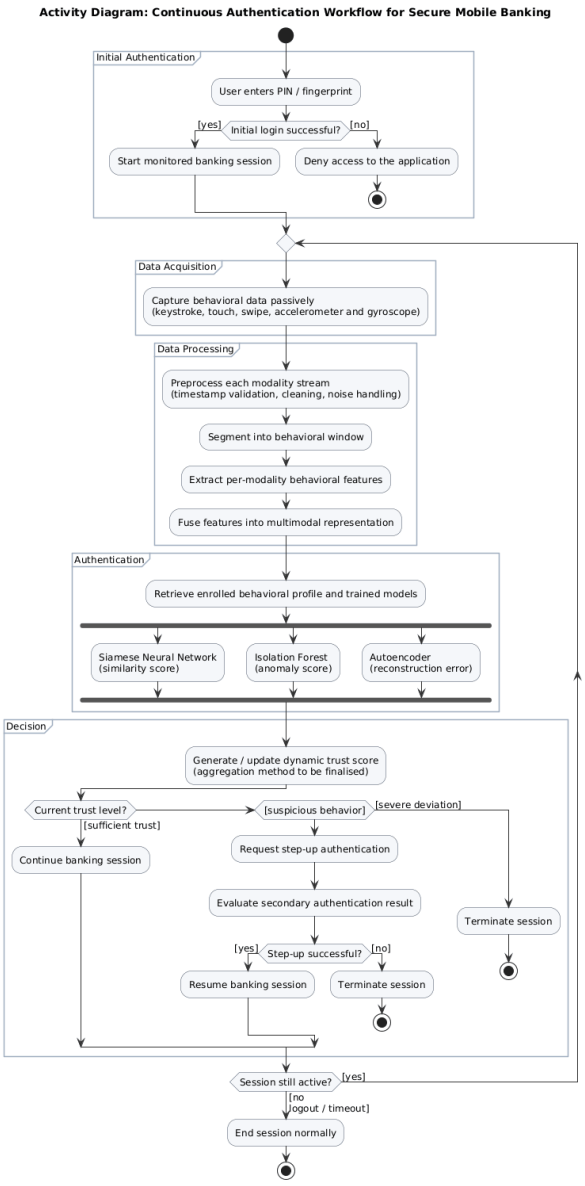


Figure 2.3: Activity diagram of the continuous authentication workflow after initial login.

Continuous authentication begins only after a successful initial login. In each cycle the application captures the four behavioral streams, which are preprocessed, segmented into windows, converted into features and fused. The fused representation is evaluated by the Siamese Neural Network, Isolation Forest and Autoencoder against the enrolled profile, and their outputs update the dynamic trust score. Three paths follow. With sufficient trust, the session continues and monitoring resumes with the next window. With suspicious behavior, step-up authentication is requested: success returns the user to the session and monitoring continues, and failure terminates the session. With severe deviation, the session is terminated. The cycle repeats until the session ends through logout or timeout. Figure 2.3 corresponds to DO2 to DO4 of Part I and refines the workflow of Part I, Figure 1.2.

2.2.3 Sequence Diagram

The sequence diagram shows the chronological exchange of messages among the user, the mobile application, the REST API, the processing pipeline, the authentication components and the profile store.

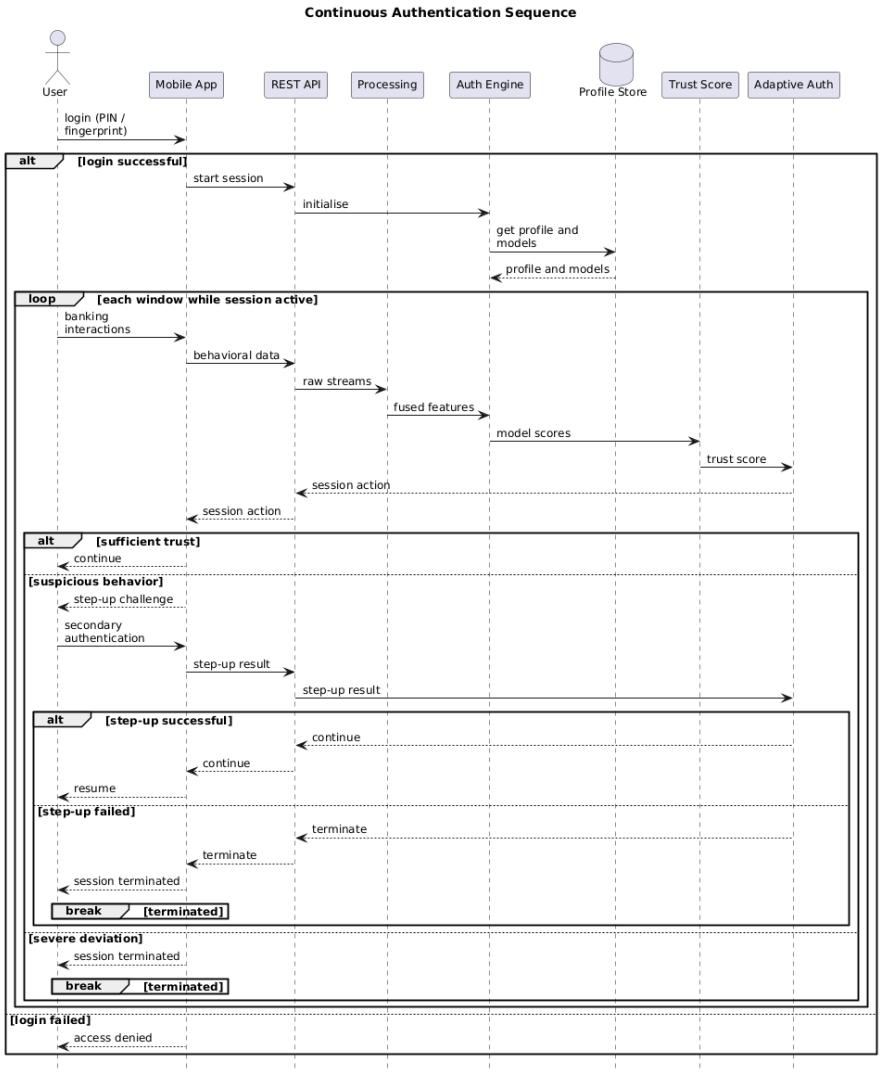


Figure 2.4: Sequence diagram of continuous authentication during an active banking session.

After a successful login the application starts a monitored session, and the Authentication Engine loads the enrolled profile and trained models from the store; the application does not access the store directly. For each behavioral window, data passes from the application through the REST API to the processing pipeline, which performs preprocessing, feature extraction and fusion. The Authentication Engine returns model scores to the Trust Score Generator, which passes the trust score to the Adaptive Authentication Engine. The resulting session action returns to the application through the API. Alternative fragments cover continuation, step-up authentication with its success and failure cases, and termination. The loop fragment represents repeated monitoring. Preprocessing, feature extraction and fusion are shown as one participant for readability. The device/backend split and the login-failure handling are not specified in Part I and are shown conservatively. Figure 2.4 corresponds to the runtime path and arrows A01 to A14 of Part I, Figure 1.3.

2.2.4 Class Diagram

A class diagram is appropriate because the system has distinct logical components with defined responsibilities and relationships. It is presented as a proposed logical structure at design stage and not as existing code.

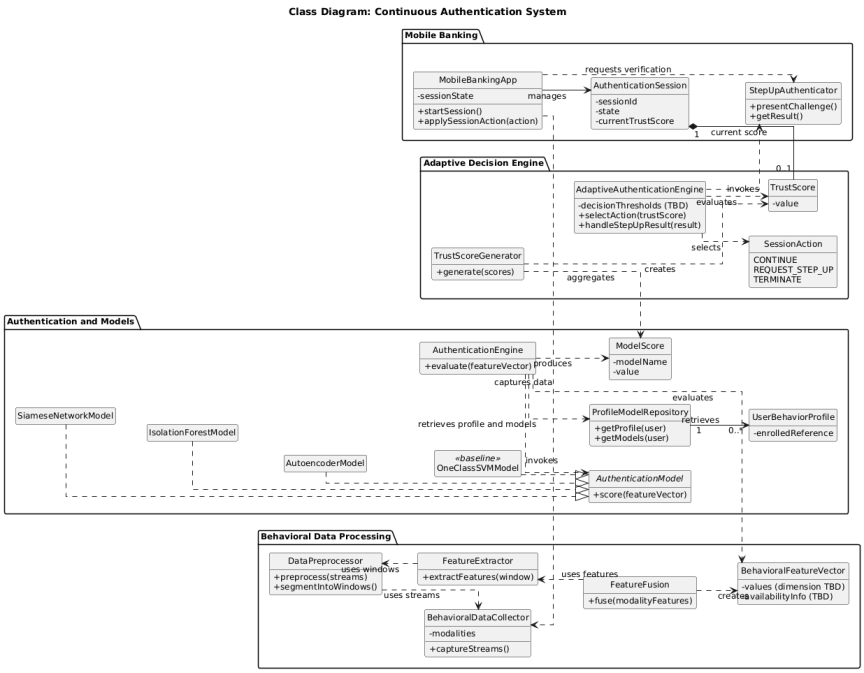


Figure 2.5: Class diagram of the proposed logical structure of the continuous authentication system.

Table 2.23: Responsibilities of the classes in Figure 2.5.

Class	**Responsibility**
MobileBankingApp	Hosts the interface, starts the session, applies session actions
StepUpAuthenticator	Presents the secondary authentication challenge and returns the result (mechanism TBD)
AuthenticationSession	Holds session state and the current trust score
BehavioralDataCollector	Captures the four behavioral streams
DataPreprocessor	Preprocesses and segments streams into windows
FeatureExtractor / FeatureFusion	Extract per-modality features and combine them into one representation
BehavioralFeatureVector	Fused representation (dimension TBD)
AuthenticationEngine	Evaluates a feature vector with the configured models
AuthenticationModel (interface)	Common scoring contract realized by SNN, IF and AE; OneClassSVMModel is the evaluation baseline
ModelScore	One model output (similarity, anomaly score or reconstruction error)
ProfileModelRepository, UserBehaviorProfile	Access to stored profiles and models; enrolled reference
TrustScoreGenerator, TrustScore	Aggregate model scores into the trust score (weighted fusion proposed in Section 2.1.5; weights TBD)
AdaptiveAuthenticationEngine	Selects the session action from the trust score and handles the step-up result (thresholds TBD)
SessionAction	Enumeration of outcomes: CONTINUE, REQUEST_STEP_UP, TERMINATE

The processing classes form a dependency chain from raw streams to the fused vector. The Authentication Engine aggregates the model implementations, which realize a common interface, and obtains profiles through the repository. Session outcomes are kept as a separate enumeration so that they are not confused with model predictions. Runtime outcome logging and raw-data storage are specified by the Database Module (Sections 2.1.6 and 2.4) and are not drawn as separate classes in this diagram.

2.3 Algorithm / Process Design

2.3.1 Enrollment and Training (Offline)

Algorithm 1: Training and Enrollment

Input: HMOG dataset D, partition plan P (Objective A or B).

Output: preprocessing artifact Π, shared SNN, per-user AE/IF, user profiles, calibration (normalization statistics, weights, α, τ_1, τ_2).

1. Import the usable HMOG users and sessions and store them with pseudonymous codes.
2. Partition D by session (Objective A) or by participant (Objective B) **before windowing**.
3. Segment sessions into windows and extract features from the raw events using the feature definitions of Part I. A modality with insufficient data in a window is marked unavailable; the Fusion Layer applies a neutral value with an availability flag (Option A in Part I, to be confirmed).
4. Fit scaling parameters (μ_j, σ_j) on the training partition only and store them as Π.
5. Normalize and fuse the features:

$$x = [x_{\text{key}} \mid x_{\text{touch}} \mid x_{\text{swipe}} \mid x_{\text{motion}}]$$

6. Build SNN pairs within the training partition only:
   - Positive pairs: non-overlapping samples from the same participant, preferably from different sessions.
   - Negative pairs: samples from different participants, preferably from the same task context.
7. Train the SNN on the training pairs, using early stopping on validation data.
8. For each user u:
   (a) $E_u \leftarrow$ enrollment samples of user u.
   (b) $r_u \leftarrow$ reference embeddings $\{f_\theta(x) : x \in E_u\}$.
   (c) Fit AE_u and IF_u using genuine samples of user u only.
   (d) Store the user profile:
$$\text{profile}(u) = (r_u, AE_u, IF_u, \text{version of } \Pi)$$
9. Using genuine and impostor scores from the validation partition:
   (a) Estimate $s_{\min}$ and $s_{\max}$, or percentile tables, for each model.
   (b) Tune w_{sim}, w_{ae}, w_{if} and α.
   (c) Choose $\tau_1 > \tau_2$ based on the FAR–FRR trade-off.
10. Fit the OC-SVM baseline using the same procedure and save all artifacts with version information.
11. Evaluate once on the test partition and report the metrics separately for Objective A and Objective B.

2.3.2 Continuous Authentication (Runtime)

During runtime, the system continuously monitors the user's behavioral patterns after the initial login. Each incoming behavioral window is preprocessed, converted into a feature representation, and evaluated using the enrolled user's models. The resulting scores are combined into a trust score, which determines whether the session continues, requires step-up authentication, or terminates.

Algorithm 2: Runtime Processing Steps

1. Capture behavioral events from keystroke timing, touch interactions, swipe gestures and device motion sensors.
2. Preprocess the incoming streams and segment them into behavioral windows.
3. Extract modality-specific features and apply the stored preprocessing parameters Π.
4. Normalize and fuse the feature vectors:

$$x = [x_{\text{key}} \mid x_{\text{touch}} \mid x_{\text{swipe}} \mid x_{\text{motion}}]$$

5. Load the enrolled user's reference embeddings, autoencoder and Isolation Forest model.
6. Calculate the similarity score, reconstruction error and anomaly score using the enrolled models.
7. Normalize and combine the scores to obtain the current trust score T_t.

8. Smooth the trust score using the previous smoothed value $\widetilde{T}_{t-1}$:

$$\widetilde{T}_t = \alpha T_t + (1-\alpha)\widetilde{T}_{t-1}$$

9. Compare the smoothed trust score with the thresholds τ_1 and τ_2.
10. Apply the corresponding session action:
    - **Continue:** proceed with the session and monitor the next window.
    - **Step-up authentication:** pause sensitive operations and request secondary authentication.
    - **Terminate:** end the session, require full re-login and record the reason.

Successful step-up authentication allows the session to continue, while failure results in termination.

Model scoring procedure. Procedure: ScoreModels

Input: fused feature vector x, enrolled user profile.

Output: similarity score s_{sim}, reconstruction error s_{ae} and anomaly score s_{if}.

1. Generate the embedding: $z = f_\theta(x)$.
2. Calculate the similarity between z and the user's reference embeddings r_u:

$$s_{\text{sim}} = \text{Similarity}(z, r_u)$$

   Similarity may be calculated using the mean or maximum similarity over the reference embeddings.
3. Reconstruct the input using the user's autoencoder:

$$\hat{x} = AE_u.\,\text{decode}\,(AE_u.\,\text{encode}(x))$$

4. Calculate the reconstruction error: $s_{\text{ae}} = \|x - \hat{x}\|^2$.
5. Calculate the Isolation Forest anomaly score: $s_{\text{if}} = IF_u.\,\text{anomaly_score}(x)$.
6. Return s_{sim}, s_{ae} and s_{if} for trust-score evaluation.

2.3.3 Trust-Score Evaluation

The trust-score evaluation module combines the outputs of the Siamese Neural Network (SNN), Autoencoder (AE) and Isolation Forest (IF) into a single trust score. The scores are normalized to a common range and oriented so that higher values indicate more trustworthy behavior. Exponential smoothing is then applied to reduce the effect of short-term variations.

Algorithm 3: ComputeTrust

Input: similarity score s_{sim}, reconstruction error s_{ae}, anomaly score s_{if} and previous smoothed trust score $\widetilde{T}_{\text{prev}}$.

Parameters: normalization statistics, weights w_{sim}, w_{ae}, w_{if} and smoothing factor α.

1. Normalize the similarity score and clip it to the range $[0, 1]$:

$$\hat{s}_{\text{sim}} = \text{Clip}_{[0,1]}\,(\text{Norm}(s_{\text{sim}}))$$

2. Normalize the reconstruction error and reverse its orientation so that a lower error gives a higher score:

$$\hat{s}_{\text{ae}} = 1 - \text{Clip}_{[0,1]}\,(\text{Norm}(s_{\text{ae}}))$$

3. Normalize the anomaly score and reverse its orientation:

$$\hat{s}_{\text{if}} = 1 - \text{Clip}_{[0,1]}\,(\text{Norm}(s_{\text{if}}))$$

4. Calculate the weighted trust score:

$$T_t = w_{\text{sim}}\hat{s}_{\text{sim}} + w_{\text{ae}}\hat{s}_{\text{ae}} + w_{\text{if}}\hat{s}_{\text{if}}$$

5. Apply exponential smoothing:

$$\widetilde{T}_t = \alpha T_t + (1-\alpha)\widetilde{T}_{\text{prev}}$$

6. Return the current trust score T_t and smoothed trust score $\widetilde{T}_t$.

The weights determine the relative contribution of each model to the overall trust score. The smoothing factor α controls how quickly the smoothed score responds to changes in behavior. The normalization statistics, weights and smoothing factor are estimated using validation data.

2.3.4 Adaptive Authentication Decision

The adaptive authentication decision module compares the smoothed trust score $\widetilde{T}_t$ against two thresholds, τ_1 and τ_2, to determine the appropriate session action. The higher threshold τ_1 represents trusted behavior, while the lower threshold τ_2 represents a level below which the session must be terminated.

Algorithm 4: Decide and ApplyAction

Input: smoothed trust score $\widetilde{T}_t$, thresholds τ_1 and τ_2, and step-up grace counter.

Output: session action.

1. If $\widetilde{T}_t \geq \tau_1$, set the action to **CONTINUE**.
2. Otherwise, if $\widetilde{T}_t < \tau_2$, set the action to **TERMINATE**.
3. Otherwise ($\tau_2 \leq \widetilde{T}_t < \tau_1$): if the step-up grace counter is still active, set the action to **CONTINUE**; if it is zero, set the action to **STEP-UP**.
4. Apply the selected action:
   - **CONTINUE:** decrement the grace counter if it is greater than zero and proceed to the next behavioral window.
   - **STEP-UP:** pause sensitive operations and request secondary authentication, such as a PIN, fingerprint or OTP (the final method is to be determined).
     - Record the step-up event and authentication result.
     - If authentication succeeds, reset the smoothed trust score to τ_1, initialize the grace counter to N windows and resume the session.
     - If authentication fails, terminate the session.
   - **TERMINATE:** end the session, require full re-login and record the termination reason.

2.3.5 Complexity and Latency Considerations

The computational cost for each behavioral window is primarily determined by feature extraction, a forward pass through the SNN encoder, reconstruction through the Autoencoder, and traversal of the Isolation Forest trees. These operations involve feed-forward neural network computations and tree-based scoring.

Authentication latency is measured from the arrival of a behavioral window to the completion of the trust-score update. Latency is evaluated for individual models and for the complete pipeline. These measurements help determine whether inference should run on-device using TensorFlow Lite or through a REST API, as discussed in Part I, Section 1.6.7.

2.4 Database Design

2.4.1 Entities, Attributes and Keys

The database is organized into entities that represent users, devices, sessions, behavioral windows, preprocessing configurations, feature vectors, trained models and authentication records. Table 2.24 lists every entity. Underlined attributes are primary keys (PK); foreign keys (FK) show the table they reference. Data types are logical and may be adapted during implementation. The attribute `dataset_id` is a source identifier (for example the HMOG release) and not a separate entity.

Table 2.24: Entities, attributes and keys.

Entity	Attributes
USER	user_id, participant_code (unique, pseudonymous), origin (HMOG / demonstration), enrollment_status, created_at
DEVICE	device_id, user_id (FK → USER), device_model, os_version, sensor_info
SESSION	session_id, user_id (FK → USER), device_id (FK → DEVICE), mode (dataset / enrollment / runtime), partition_rule (train / validation / test), start_time, end_time, raw_data_ref

Continued on next page

Table 2.24: Entities, attributes and keys (continued).

Entity	Attributes
BEHAVIORAL_ WINDOW	window_id, session_id (FK → SESSION), window_index, task_context, start_time, end_time, modality_availability, quality_flag
PREPROCESSING_ ARTIFACT	artifact_id, version, scaling_method, missing_value_parameters, windowing_config, feature_config, created_at
FEATURE_VECTOR	feature_id, window_id (FK → BEHAVIORAL_WINDOW), preprocessing_id (FK → PREPROCESSING_ARTIFACT), dataset_id, feature_values
MODEL_ARTIFACT	model_id, model_type (SNN / AE / IF / OCSVM baseline), scope (shared / per-user), user_id (FK → USER, nullable if shared), preprocessing_id (FK → PREPROCESSING_ARTIFACT), version, hyperparameters, model_file_ref, validation_metrics, created_at
USER_PROFILE	profile_id, user_id (FK → USER), snn_model_id (FK → MODEL_ARTIFACT), ae_model_id (FK → MODEL_ARTIFACT), if_model_id (FK → MODEL_ARTIFACT), enrollment_status, created_at, updated_at
PROFILE_REFERENCE	reference_id, profile_id (FK → USER_PROFILE), vector_ref, feature_id (FK → FEATURE_VECTOR), created_at
DECISION_CONFIG	config_id, version, w_sim, w_ae, w_if, alpha, tau1, tau2, normalization_ref, calibration_data, created_at
MODEL_SCORE	score_id, window_id (FK → BEHAVIORAL_WINDOW), model_id (FK → MODEL_ARTIFACT), raw_score, normalized_score, score_type, created_at
TRUST_RECORD	trust_id, window_id (FK → BEHAVIORAL_WINDOW), user_id (FK → USER), raw_score, normalized_score, trust_score, previous_trust_score, decision_config_id (FK → DECISION_CONFIG), created_at
DECISION_LOG	decision_id, trust_id (FK → TRUST_RECORD), window_id (FK → BEHAVIORAL_WINDOW), decision, action, reason, created_at
STEPUP_EVENT	stepup_id, decision_id (FK → DECISION_LOG), method, result (success / failure), response_time, created_at
SIAMESE_PAIR	pair_id, feature_id_1 and feature_id_2 (FK → FEATURE_VECTOR), pair_label, similarity_relation, partition_rule (train / validation / test), created_at
EVALUATION_RUN	run_id, model_id (FK → MODEL_ARTIFACT, nullable for fixed run), config_id (FK → DECISION_CONFIG), objective, dataset_id, created_at
EVALUATION_METRIC	metric_id, run_id (FK → EVALUATION_RUN), metric_name (FAR, FRR, EER, Precision, Recall, F1, ROC-AUC, latency), value

2.4.2 Relationships

Relationships follow the foreign keys in Table 2.24 and the cardinalities shown in Figure 2.6. Most are one-to-many (for example USER–SESSION and SESSION–BEHAVIORAL_WINDOW). The special cases are:

- BEHAVIORAL_WINDOW–FEATURE_VECTOR, BEHAVIORAL_WINDOW–TRUST_RECORD and TRUST_RECORD–DECISION_LOG are one-to-one.
- DECISION_LOG–STEPUP_EVENT is one-to-zero-or-one, since only step-up decisions have an event.
- USER_PROFILE–FEATURE_VECTOR is many-to-many through PROFILE_REFERENCE.
- USER_PROFILE–MODEL_ARTIFACT has three references (SNN, AE, IF).

2.5 User Interface Design

This section describes the proposed user interface of the mobile banking application prototype (C1, B01) and the Secondary Authentication Handler (C9, B02) introduced in Part I, Section 1.4. The planned screens support the simulated banking tasks and the three session actions of the Decision layer: continue, request step-up authentication, and terminate. Since implementation has not yet started, this section



Figure 2.6: Entity–relationship diagram of the continuous authentication system.

presents the interface requirements and planned screen layouts in textual form. Detailed UI mockups will be prepared during the implementation phase.

2.5.1 Design Principles

- **Passive monitoring:** behavioral capture runs in the background, and the legitimate user sees no additional prompt during normal use.
- **Familiar banking-style layout:** the screens support the tasks defined in Part I (login, fund-transfer simulation, balance inquiry and navigation) so that interaction resembles ordinary mobile banking.
- **Graded response:** the interface presents the three outcomes defined in Part I (continue, step-up authentication, terminate) as three distinct visible states.
- **Data minimization:** no screen stores typed characters, PINs, passwords, account numbers or entered amounts (Part I, Section 1.5.7). The application is a simulated one and uses no real banking data.
- **No internal details exposed:** model scores, fusion weights and thresholds are not shown to the end user.

2.5.2 Screen Overview

Table 2.25 summarizes the proposed screens and their relation to the architecture. Arrow identifiers refer to Part I, Table 1.4.

Table 2.25: Proposed screens of the mobile banking application prototype.

Screen	Purpose	Main elements (proposed)	Related items
Login	Initial authentication; starts the monitored session	PIN entry or fingerprint prompt; login action	B00, B01; A01
Simulated banking dashboard	Hosts the banking tasks during which behavior is captured	Balance inquiry view; fund-transfer simulation form; navigation controls	B01, B04a–d; A02
Authentication status	Indicates the state of the monitored session	Verification-required notice; session-ended notice; optional evaluator demonstration view (TBD); internal trust score not shown to the user	B01; A09, A10
Step-up authentication	Secondary challenge when behavior is suspicious	Notice of additional verification; input area for the challenge (mechanism TBD); submit action	B02; A10, A11
Session termination	Informs the user that the session has ended	Termination notice; action to return to login	B01; A09

2.5.3 Login Screen

The login screen is the entry point of the prototype. The user completes the existing PIN or fingerprint login, which is not part of the continuous authentication models. A successful login generates the session-start event (A01) and begins the monitored session. Behavioral monitoring starts only after this event. Entered PINs are not stored, and keystroke capture records press and release timing only (Part I, Section 1.5.1).

2.5.4 Simulated Banking Dashboard

The dashboard is the main working screen and provides access to the simulated banking tasks: balance inquiry, fund-transfer simulation and navigation. It is the screen on which touch, swipe, keystroke and device-motion data are generated during use. Numeric entry in the fund-transfer form uses the proposed in-app keypad (Section 2.1.1; to be confirmed by the project guide). These events are forwarded passively to the Data Acquisition Module (A02) without a deliberate action from the user. No real accounts or transactions are involved.

2.5.5 Authentication Status Display

During normal operation, continuous authentication runs in the background without displaying the internal trust score, model outputs or decision thresholds to the user. When additional verification is required or the session is terminated, the interface displays the appropriate notification. For demonstration purposes, a separate view showing the trust score over time may be provided to evaluators. The inclusion of this demonstration view and its final layout remain TBD. The trust-score scale and decision thresholds are not defined here (Part I, Section 1.6.4).

2.5.6 Step-Up Authentication Screen

When the Adaptive Authentication Engine requests secondary authentication (A10), the Secondary Authentication Handler presents this screen and interrupts the current task. It informs the user that additional verification is required and provides an input area for the challenge. The outcome is returned as success or failure (A11): on success the session continues, and on failure the session is terminated. The challenge mechanism is **TBD** (Part I, Section 1.4.1, component C9) and is not assumed here.

2.5.7 Session Termination Screen

The session termination screen is shown when the Decision layer returns a terminate action (A09), either because the deviation is severe or because step-up authentication has failed. It informs the user that the session has ended and returns the application to the login state, so that a new session requires a fresh login. The wording of the message is **TBD**.

2.5.8 Collection-Mode Screens

The same prototype is proposed for optional pilot and demonstration sessions (Part I, Section 1.5.1). In collection mode the application would show a consent screen before recording, stating what is recorded and for what purpose (Part I, Section 1.5.7), and would assign a pseudonymous participant code to each session. These screens are not part of the runtime authentication flow, and their layout is **TBD**.

2.5.9 Status and Open Items

The user interface is currently at the design stage, and no screens have been implemented or evaluated. Detailed mockups, the step-up authentication mechanism, the visibility of the authentication status indicator, the wording of user messages and the layout of the collection-mode screens will be finalized during implementation.

2.6 Hardware and Software Design

This section lists the hardware and software required by the system described in Part I. Each item carries the status markers of Part I: **C** (confirmed in the approved Extended Abstract), **P** (proposed in this design, to be confirmed during implementation) and **U** (unresolved). Unspecified values are marked **TBD**. No hardware specification, software version or library has been selected, and nothing in this section has been implemented or verified.

2.6.1 Hardware Design

The hardware requirements are summarized in Table 2.26. Behavioral data are collected from Android devices that may differ in their available sensors and operating-system versions (Part I, Section 1.5.7). The `device_descriptor` field in Part I, Table 1.7 records the device model and OS version to support interpretation of the collected data. The system does not require a dedicated fingerprint sensor for continuous authentication; the initial fingerprint login, if supported by the device, uses the device's existing authentication facility. The behavioral authentication process relies on the approved touch, keystroke and device-motion features.

Table 2.26: Hardware requirements.

ID	Item	Requirement / role	Status
HW1	Android smartphone	Runs the prototype banking-style application and captures behavioral data in pilot and demonstration sessions. Device models, OS versions and minimum specifications are TBD.	C (platform), TBD (models)
HW2	Touchscreen	Provides touch and swipe pointer events (coordinates, timing).	C
HW3	Touch pressure support	Pressure is used as a touch and swipe feature only if the device exposes it. Availability depends on the device.	C (listed), U (availability)
HW4	Accelerometer	Provides three-axis device-motion readings. Sampling rate, units and frame are TBD.	C (readings), U (units)
HW5	Gyroscope	Provides three-axis rotation readings. Sampling rate, units and frame are TBD.	C (readings), U (units)
HW6	Keypad input	Provides key press and release timing for keystroke dynamics through the proposed in-app keypad, because standard soft keyboards generally do not expose release events (Section 2.1.1). Only timing is recorded, not typed characters (Part I, Section 1.5.1).	C (timing), P, U (keypad)
HW7	Development workstation	Used for the offline path: training and evaluation of the models and storage of the dataset and model artifacts. Specifications, including any GPU, are TBD.	P, TBD (specifications)

2.6.2 Software Design

This subsection describes the software components required to support the proposed behavioral biometric-based continuous authentication system. The software design includes the mobile banking application prototype, behavioral data acquisition and preprocessing modules, machine learning models, trust-score generation, adaptive authentication, secondary authentication and the REST API interface. The selected software versions, frameworks, libraries and storage technology remain TBD and will be finalized during implementation. The software requirements and their implementation status are summarized in Table 2.27.

2.6.3 Deployment Arrangement

Part I does not fix where authentication runs. Table 2.28 lists the two options under consideration. The choice affects the payload of the REST API, the inference runtime and the measured latency, and is not made in this document.

The dashed backend enclosure in Part I, Figure 1.3 may change in extent depending on this decision. Authentication latency will be measured for each model and for the complete pipeline (Part I, Section 1.6.7) and will inform the choice.

2.6.4 Unresolved Implementation Choices

The hardware and software decisions that remain unresolved, together with their reasons and related requirement identifiers, are summarized in Table 2.29.

2.6.5 Status

This section states requirements and options only. The Extended Abstract is stated in Part I to list candidate software. These candidates are not reproduced here and are treated as unselected until adopted. No version or specification is claimed, and no implementation or performance result is reported.

Table 2.27: Software requirements.

ID	Item	Requirement / role	Status
SW1	Android operating system	Target platform of the mobile prototype. Versions are TBD.	C (platform), TBD (versions)
SW2	Mobile banking application prototype (C1)	Provides the simulated banking tasks, session handling and collection of behavioral events (Section 2.5).	C (prototype), P (design)
SW3	Data acquisition and collection mode (C2)	Captures and timestamps the four behavioral streams, and labels pilot and demonstration sessions with pseudonymous codes in collection mode.	P
SW4	Preprocessing, feature extraction and fusion (C3–C5)	Implements the pipeline of Part I, Sections 1.5.4 and 1.6.2. The same definitions are used in training and at runtime. Implementation language and libraries are TBD.	P, TBD (tools)
SW5	Authentication models (C6)	Siamese Neural Network, Autoencoder and Isolation Forest, with a One-Class SVM used only as a comparison baseline. Framework and libraries are TBD.	C (models), TBD (tools)
SW6	Trust score generator and adaptive authentication engine (C7, C8)	Aggregates model outputs and applies the thresholds to select the session action. Weights and thresholds are TBD (Section 2.1.5).	P
SW7	Secondary authentication handler (C9)	Presents the step-up challenge. The mechanism is TBD.	P, U (mechanism)
SW8	REST API interface (C10)	Connects the application to the backend machine learning engine. Payload and framework are TBD.	P
SW9	Data and model storage (C11, C12)	Holds the behavioral dataset, enrolled profiles, trained models and preprocessing parameters. The storage technology is TBD (Part I, Section 1.5.7).	P, U (technology)
SW10	Evaluation module (C14)	Computes FAR, FRR, EER, Precision, Recall, F1-score and latency. ROC-AUC, the confusion matrix and accuracy are reported as additional measures (Part I, Section 1.6.7).	C (core metrics), P (module, additional metrics)
SW11	On-device inference runtime	TensorFlow Lite is mentioned in Part I as a possible option for on-device inference, but the decision is not made.	U

Table 2.28: Open deployment options (unresolved).

Option	Description	Dependent items
Backend inference	The application sends requests through the REST API, and the backend machine learning engine returns the result.	SW8, SW9, latency measurement
On-device inference	Selected processing stages run directly on the mobile device. TensorFlow Lite is a possible runtime.	SW11, SW4, latency measurement

Table 2.29: Hardware and software items that remain to be decided.

Item	Reason	Related items
Android device models and OS versions	Depends on pilot and demonstration devices and their availability	HW1, SW1
Sensor sampling rates, units and time base	Not specified in the approved documents	HW4, HW5
Programming languages, frameworks and libraries	Not selected	SW4, SW5, SW8
Storage technology and file formats	No data-storage decision is adopted	SW9
Workstation specifications	Depend on dataset size and model size, both TBD	HW7
Step-up challenge mechanism	Not specified	SW7
On-device versus backend inference	To be decided after latency considerations	SW8, SW11